resumer

Développement mobile

USTHB · Ingénieur 5 · Software Engineering · S9

Définitions · Comparaisons · Méthodes · Révision active

Synthèse des documents locaux disponibles au 7 novembre 2026. Enseignante du chapitre 2 : Dr Kouchi. Le chapitre 1 n'est fourni que par un lien Prezi non accessible lors de la préparation : son contenu n'est donc pas reconstitué. Le résumé couvre le chapitre 2, le TD1 et les TP1 à TP3.

Mise à jour : 11 novembre 2026

Sommaire

Chapitre 2 — Appareils mobiles et architecture matérielle 3

Chapitre 2 — Écrans, résolution, densité et unités Android 3

TD1 — Méthode de résolution des exercices 4

TP1 — Projet Android et cycle de vie 5

TP2 — Interface XML et calcul du double 6

TP3 — Deux activités, Intent et transmission de données 7

Révision active et sources 7

Chapitre 2 — Appareils mobiles et architecture matérielle

Appareil mobile et SoC

Un appareil mobile est un système portable fonctionnant avec une alimentation autonome. L'évolution PDA → téléphone multifonction → smartphone rassemble téléphonie, applications, capteurs et communications dans un même appareil.

Le **SoC, System on Chip**, intègre de nombreuses fonctions de calcul et de communication sur une puce. Il favorise la compacité, les échanges rapides et la maîtrise de la consommation. Attention : toutes les mémoires et tous les capteurs d'un téléphone ne sont pas nécessairement intégrés physiquement au SoC.

Élément	Fonction essentielle	À distinguer
CPU	Exécuter les instructions et la logique générale	Nombre de cœurs et fréquence ne suffisent pas à mesurer toute la performance
GPU	Traiter le rendu graphique et des calculs parallèles	Ne remplace pas le CPU pour toutes les tâches
DSP	Effectuer des traitements spécialisés de signaux	Audio, image ou communications selon le matériel
RAM	Conserver temporairement les données des programmes actifs	Volatile : contenu perdu à l'arrêt
Stockage flash	Conserver OS, applications et fichiers	Non volatile ; souvent appelé « ROM » par abus de langage
Modem / interfaces radio	Assurer les communications	Distincts des fonctions de calcul général

Une mémoire externe, par exemple microSD, est optionnelle. Le cours présente aussi les processeurs spécialisés pour l'IA : il faut retenir leur fonction plutôt que mémoriser des noms commerciaux ou des performances rapidement périmées.

Contraintes à relier au logiciel

La batterie, la mémoire disponible, la puissance de calcul et la diversité des écrans influencent la conception. Une application doit rester lisible sur différentes densités et éviter de conserver inutilement des ressources lorsque son activité n'est plus utilisée.

Chapitre 2 — Écrans, résolution, densité et unités Android

Les notions à ne pas confondre

Notion	Signification	Exemple
Pixel, px	Point de l'image affichée	Un élément de 200 px
Résolution	Largeur × hauteur en pixels	1080 × 1920 px
Diagonale physique	Taille de l'écran en pouces	4 pouces ; 1 pouce = 2,54 cm
PPI	Nombre physique de pixels par pouce	Densité calculée à partir de la résolution et de la diagonale

Notion	Signification	Exemple
dp / dip	Unité logique indépendante de la densité	Largeur, hauteur et marges d'une vue
sp	Unité de taille de texte adaptée aux préférences utilisateur	Texte à 16 sp

Une résolution plus élevée n'implique pas une surface physique plus grande. À nombre de pixels identique, une forte densité réduit la longueur physique occupée. Android utilise une densité logique pour convertir les dp ; elle peut différer du PPI physique mesuré. Dans les exercices, on assimile ces valeurs lorsque l'énoncé le demande.

Formules indispensables

```
Diagonale_pixels = sqrt(largeur_px² + hauteur_px²)
PPI = Diagonale_pixels / diagonale_pouces
px = dp × dpi / 160
dp = px × 160 / dpi
diagonale_pouces = Diagonale_pixels / PPI
```

À la densité de référence de 160 dpi, 1 dp correspond à 1 px. Le dp conserve une taille visuelle approximativement cohérente ; il ne promet pas une longueur physique absolument identique sur tous les appareils.

Classe	Densité de référence	Multiplicateur px / dp
ldpi	120 dpi	0,75
mdpi	160 dpi	1
hdpi	240 dpi	1,5
xhdpi	320 dpi	2
xxhdpi	480 dpi	3
xxxhdpi	640 dpi	4

Ces classes sont des références de ressources ; un PPI physique calculé n'est pas forcément égal à l'une d'elles. Pour du code réel, employer les unités XML et les fonctions Android de conversion. Référence complémentaire : Android — densités et unités.

Exemple de mise en page

```
<Button
    android:layout_width="120dp"
    android:layout_height="48dp"
    android:text="Valider"
    android:textSize="16sp" />
```

La largeur occupe 120 px à 160 dpi, 240 px à 320 dpi et 360 px à 480 dpi. Le texte est en sp pour respecter les préférences de taille de police.

TD1 — Méthode de résolution des exercices

Procédure

1. Identifier les données : résolution, diagonale, dimensions physiques ou densité.

2. Garder la même orientation pour largeur et hauteur dans toutes les unités.

3. Calculer la densité manquante, puis convertir chaque dimension.

4. Vérifier les unités et arrondir seulement à la fin.

5. Distinguer une densité exacte calculée d'une classe Android approximative.

Exemples calculés

Énoncé	Calcul	Résultat
200 px à 320 dpi	200 × 160 / 320	100 dp
120 dp à 240 dpi	120 × 240 / 160	180 px
480 × 800 px et 600 × 1000 dp	480 × 160 / 600, confirmé par l'autre dimension	128 dpi
dp = 4 × px	4 = 160 / dpi	40 dpi, cas théorique
720 × 1280 px et 360 × 640 dp	720 × 160 / 360	320 dpi
1280 × 720 px, diagonale 4 pouces	sqrt(1280² + 720²) / 4	Environ 367,15 PPI

Pour le dernier exemple, les dimensions logiques obtenues en assimilant dpi et PPI sont environ **557,81 × 313,77 dp**. Le corrigé local indique une diagonale de 1452,67 px, mais le calcul donne environ **1468,60 px** ; les résultats qui en dépendent ont été rectifiés ici.

Pour une icône de 0,5 pouce dans le modèle de l'exercice : taille_px = 0,5 × dpi. On obtient 60, 80, 120, 160, 240 et 320 px dans l'ordre ldpi à xxxhdpi, soit 80 dp dans ce modèle de référence.

Si les dimensions physiques horizontale et verticale sont données séparément, vérifier la cohérence des rapports pixels/pouces sur les deux axes. Un simple quotient de diagonales peut masquer des données incompatibles. Le corrigé du TD contient aussi une variante d'énoncé et des inversions d'orientation : toujours repartir de la question originale.

TP1 — Projet Android et cycle de vie

Organisation du projet

Le TP crée un projet **Empty Views Activity**, en Java ou Kotlin, avec une interface XML. Gradle construit le projet et résout ses dépendances ; le SDK fournit les outils et API Android ciblés.

Élément	Rôle
AndroidManifest.xml	Déclarations de l'application et de ses composants, dont les activités
MainActivity.java	Comportement de l'activité
res/layout	Description XML des vues
res/drawable	Images et ressources graphiques
res/mipmap	Icônes de lancement
res/values	Chaînes, couleurs, styles et autres valeurs
res/xml	Configurations XML selon leur usage

`setContentView(R.layout.activity_main)` associe l'activité à son interface. L'intent-filter avec MAIN et LAUNCHER désigne l'activité de lancement. Les ressources audio ne sont pas toutes à placer dans drawable : des fichiers bruts peuvent relever de res/raw. Référence : Android — organisation des ressources.

Cycle de vie à mémoriser

```
Premier affichage : onCreate → onStart → onResume
Perte du premier plan : onPause
Activité devenue invisible : onStop
Retour après arrêt : onRestart → onStart → onResume
Destruction normale de l'instance : onDestroy
```

`onCreate` initialise l'activité ; `onStart` correspond à sa visibilité ; `onResume` à son interaction au premier plan. `onPause` et `onStop` permettent d'adapter les traitements au changement d'état. `onDestroy` n'est pas garanti si le processus est tué : ce n'est pas un point fiable unique pour enregistrer des données importantes. Une notification n'entraîne pas systématiquement onPause ; le comportement dépend de l'interaction et de la visibilité. Référence : Android — cycle de vie.

Le TP observe ces transitions via `Log.d` et Logcat. Un émulateur exige une configuration compatible ; un téléphone réel avec débogage USB autorisé constitue une autre voie d'exécution. Les observations d'un appareil ne remplacent pas les règles générales du cycle de vie.

TP2 — Interface XML et calcul du double

Éléments d'interface

Le TP construit une application qui lit un nombre et affiche son double. Le layout XML décrit les vues ; l'activité Java définit le comportement.

Élément	Fonction dans le TP
LinearLayout / LinearLayoutCompat	Organiser les enfants verticalement ou horizontalement
TextView	Afficher titre, libellé ou résultat
EditText	Recueillir la saisie
Button	Déclencher le calcul
Identifiant XML	Relier une vue à `R.id...` dans Java
colors.xml	Centraliser une couleur réutilisable par `@color/result`

`match_parent` occupe l'espace offert par le parent ; `wrap_content` dimensionne selon le contenu. Garder dp pour les dimensions et sp pour le texte. Le champ `inputType="numberDecimal"` oriente la saisie, mais ne remplace pas toutes les validations de la valeur.

Du clic au résultat

Après `setContentView`, `findViewById` récupère les vues du layout. Le TP associe le clic à `public void computeDouble(View view)` via `android:onClick="computeDouble"`.

```
Clic → lire getText().toString().trim()
     → refuser une chaîne vide
     → convertir en double
     → calculer 2 × valeur
     → afficher avec TextView.setText
```

Un `Toast` informe l'utilisateur d'une saisie manquante. Le code fourni vérifie le vide ; une conversion peut encore lever `NumberFormatException` pour une chaîne invalide. Une application robuste doit aussi vérifier la

conversion et les résultats numériques. Cet ajout est un point de vigilance, pas une fonctionnalité déjà présente dans le PDF.

TP3 — Deux activités, Intent et transmission de données

Organisation

L'application d'exercice « My Health Coach » contient une activité de saisie et une activité de résultat. La première lit une masse en kg et une taille en cm ; elle calcule un indice puis le transmet à l'autre activité. Le mot « width » du document doit être compris ici comme **weight**, la masse, et non une largeur.

```
taille_m = taille_cm / 100
indice = masse_kg / (taille_m × taille_m)
```

Exemple purement numérique : 75 kg et 180 cm donnent 75 / 1,8² ≈ **23,15**. Cet exemple sert à étudier les conversions et le transfert de données dans l'application ; il ne constitue pas une interprétation médicale.

Écouteur et Intent explicite

Le TP2 reliait le clic dans XML ; le TP3 utilise aussi un écouteur Java : `computeButton.setOnClickListener(this::compute)`. Les vues doivent être initialisées avant l'installation de l'écouteur.

```
Intent intent = new Intent(this, ResultActivity.class);
intent.putExtra("EXTRA_BMI", indice);
startActivity(intent);
```

`Intent` désigne ici explicitement l'activité à ouvrir. L'extra associe une clé à une valeur ; la même clé et un type compatible doivent être utilisés à la réception.

```
double indice = getIntent().getDoubleExtra("EXTRA_BMI", 0.0);
```

Le TP lit cet extra dans `onStart`, après l'initialisation des TextView dans `onCreate`, puis formate l'affichage à deux décimales. La valeur par défaut 0,0 ne démontre pas qu'un extra était présent : il faut prévoir le comportement si la donnée manque.

Validation, suppression et navigation

Le bouton Delete vide les champs avec `setText("")`. Avant de calculer, vérifier des chaînes non vides, une conversion correcte et des valeurs strictement positives, notamment la taille pour éviter une division par zéro.

Le bouton Back du PDF démarre une nouvelle MainActivity. Cela peut empiler une autre instance ; pour simplement revenir à l'activité de saisie déjà présente sous l'activité résultat, `finish()` correspond au comportement de fermeture de l'écran courant.

Le sujet annonce un affichage d'interprétation, mais le code visible de ResultActivity met seulement à jour le texte numérique : `interpretationText` est initialisé sans règle d'interprétation dans l'extrait fourni. Il ne faut pas présenter cette fonctionnalité comme déjà implémentée.

Révision active et sources

Questions avec réponses courtes

1. **Pourquoi éviter px pour les dimensions d'une interface ?** La taille apparente varierait trop avec la densité ; utiliser dp.

2. **Une mémoire dite « ROM » de smartphone est-elle généralement une ROM immuable ?** Non : il s'agit habituellement de stockage flash réinscriptible.

3. **Que vaut 48 dp à 480 dpi ?** 144 px.

4. **Une activité peut-elle être invisible sans être détruite ?** Oui, elle peut être arrêtée et reprendre ensuite.

5. **Pourquoi compléter les logs du TP avec la documentation ?** Certaines transitions dépendent du scénario ; onDestroy n'est notamment pas garanti à la mort du processus.

6. **Quand utiliser findViewById ?** Après avoir chargé le layout contenant la vue avec setContentView.

7. **Pourquoi la taille est-elle divisée par 100 dans le TP3 ?** Pour convertir les centimètres en mètres avant le carré.

8. **Quel lien entre putExtra et getDoubleExtra ?** Ils transmettent puis récupèrent une valeur avec une clé commune et un type compatible.

9. **Le code du TP3 calcule-t-il l'interprétation annoncée ?** Pas dans l'extrait fourni : seul l'indice numérique est affiché.

Documents utilisés

• `dev-mobile/COURS/Chapter2-Resolution-Density-Android.pdf` : 35 pages ; matériel p. 3–12, affichage p. 13–33.

• `dev-mobile/TD/TD1.pdf` et `TD1-sol.pdf` : conversions et exercices ; recalcul des valeurs signalées.

• `dev-mobile/TP/TP1.pdf` : 6 pages ; structure du projet et cycle de vie.

• `dev-mobile/TP/TP2.pdf` : 4 pages ; vues XML, clic, saisie et calcul du double.

• `dev-mobile/TP/TP3.pdf` : 8 pages ; deux activités, conversion d'unités, Intent et extras.

• `dev-mobile/COURS/LINKS.md` : lien Prezi du chapitre 1 ; chapitre non intégré à cette fiche.

Les références Android servent à préciser les points ambigus du support. Les exemples numériques de cette fiche sont recalculés ; ce document ne prétend pas couvrir le chapitre 1 manquant.